

ゆっくり学ぼう、圏論から見るプログラミング入門

ぼっちまん @forget1900

2026 年 5 月 21 日

目次

1	はじめに	3
2	有向グラフと型の合成	5
3	関手と自然変換	7
4	表現可能関手と極限	9
5	デカルト閉圏とプログラミング	11
6	トポス理論との関係	15
6.1	内部論理	16
6.2	前層トポスと表現可能関手	16
7	Cubical Agda：トポスの発想に近い言語	17
7.1	等号をパスとして扱う	17
7.2	univalence	17
7.3	高次帰納型	17

※ 追加で作りました。

1 はじめに

霊夢：ゆっくり霊夢です。

魔理沙：ゆっくり魔理沙だぜ!今日は圏論について話していくぜ。霊夢は圏論について何か知っていることはあるか？

霊夢：ケンロン？あれでしょ、ワンちゃんを正しく飼うための理論とか、そういうやつでしょ？

魔理沙：それは犬論になるだろ！圏論っていうのは、数学の分野の一つで、英語では「Category Theory」と呼ばれるものなんだぜ。

霊夢：カテゴリーってことは、何かを分類するような話なの？

魔理沙：いい線いってるぜ。圏論は、数学のいろんな分野、例えば代数学や幾何学なんかの「構造」や「関係性」を、高い視点からまとめて俯瞰しようとする学問なんだ。

霊夢：高い視点？ドローンでも飛ばして数学を見下ろすの？

魔理沙：数学的な視点の高さだぜ。例えば、「集合」や「関数」なんかも、圏論の視点から見ると、ある特定の「対象（オブジェクト）」と「射（モルフィズム）」という関係で表すことができるんだ。

霊夢：対象と射？なんだか弓道みたいな言葉が出てきたわね。

魔理沙：対象っていうのは、集合や空間といった「もの」のことだ。そして射っていうのは、その「もの」同士の関係性や変換を表す矢印のことだぜ。

ここで圏論の「圏」をしっかりと定義しておこう。

定義 1 (圏の構造). 圏は以下のデータからなる。

- 対象の集まり $A, B, C, \dots$
- 射の集まり $f, g, h, \dots$
- 射の合成: $f: A \rightarrow B$ と $g: B \rightarrow C$ に対し、 $g \circ f: A \rightarrow C$ が存在する。
- 恒等射: 各対象 A に対し、 $id_A: A \rightarrow A$ が存在する。

定理 1 (圏の公理). 先の定義で紹介した要素を全て含み、かつ以下の公理を満たす構造を圏と呼ぶ。

- **結合律:** $h \circ (g \circ f) = (h \circ g) \circ f$

• 単位律: $f \circ id_A = f = id_B \circ f$

注意. 「対象の集まり」「射の集まり」は、常に集合であるとは限らない。

霊夢：矢印で関係性を表すのね。でも、それって何が便利なの？

魔理沙：ものすごく便利なんだぜ。例えば、代数学で成り立つ法則が幾何学でも同じように成り立つことを、図式として説明できたりするんだ。

霊夢：なるほどね。圏論がどんなものか、なんとなくイメージできたわ。でも、圏論はそれ以外にも活躍しているの？

魔理沙：お、珍しく意欲的じゃないか。実は私たちの生活に欠かせない『プログラミング』の世界で、圏論の考え方が大活躍しているんだ。

霊夢：プログラミング？数学の抽象的な話が、どうしてコンピューターのプログラムに関係してくるのよ？

魔理沙：特に『関数型プログラミング』と呼ばれるプログラミング言語のジャンルでは、圏論の概念がそのまま使われていることが多いんだぜ。まずは最もシンプルな例から見よう。

定義 2 (Boolean 関数プログラム). 以下の規則で構成されるプログラムを定める。

1. 変数記号 (x など)、 T (真)、 F (偽) はプログラムである。
2. P がプログラムならば、 $\neg P$ もプログラムである。
3. P_1, P_2 がプログラムならば、 $P_1 \wedge P_2$ もプログラムである。
4. 変数記号 x とプログラム P_1, P_2 について、 $\text{let } x \text{ be } P_1 \text{ in } P_2$ はプログラムである。

注意. 4. は論理的には命題 P_2 内の変数 x を別の命題 P_1 で置き換えることを意味し、プログラミングでは「代入」と呼ばれる操作に相当する。

魔理沙: Boolean っていうのは『真 (True)』と『偽 (False)』の2つの値だけで計算する、論理演算やプログラミングの基礎になる考え方のことだぜ。

霊夢：ブーリアン？ なんだか美味しそうな名前ね。イタリア料理のメニューにありそうだわ。

魔理沙：それはブイヤベースだろ！話を戻すと、 T と F は True と False のことだぜ。

霊夢：変数記号の x っていうのは、数学の授業で出てくるあれ？

魔理沙：そうだ。まだ値が決まっていない数のことだな。プログラミングではよく空箱みたいに喩えることが多いぜ。

霊夢：ふーん。でも、2つ目の『 $\neg$ 』って記号はなんだか階段の段差みたいね。

魔理沙：これは『否定 (NOT)』を表す記号だ。もし P が T (真) なら、 $\neg P$ は F (偽) になる。逆に P が F なら T になるんだ。

霊夢：続いて3つ目の『 $\wedge$ 』はテントみたいな形ね。

魔理沙：これは『かつ (AND)』を表す記号だ。 P_1 と P_2 が両方とも T の時だけ、全体が T になるんだぜ。

霊夢：最後の4つ目が、一番プログラミングっぽい構造ね。let って英語の授業で習ったわ。

魔理沙：ここでは『 P_1 というプログラムの結果を x という変数に代入して、それを P_2 というプログラムの中で使う』という意味だ。現代のコンピューターの複雑な計算も、根本を辿ればこういう真と偽の組み合わせや、値の代入のルール積み重ねで動いているんだぜ。

2 有向グラフと型の合成

魔理沙：さて、圏の構造についてももう少し深く学んでいくぜ。『点と矢印』というイメージを数学的にしっかり扱うために、有向グラフという概念を導入するぜ。

ちなみに圏論の文脈では、この有向グラフのことをクイバー (Quiver) と呼んだりもするんだ。

霊夢：クイバー！ なんだか急にかっこよくなったわね！

定義 3 (有向グラフ). 有向グラフとは、頂点集合 V と辺集合 E 、および始点・終点を与える写像 $s, t: E \rightarrow V$ からなる組 (V, E, s, t) のことである。

注意. 圏では、対象を頂点、射を辺とみなせるため、圏は有向グラフに合成と恒等射を追加した構造だと考えることができる。

霊夢：ただの点と矢印から、どうやって圏の条件である「恒等射」や「合成」を作り出すの？

魔理沙：矢印を辿ってできる「道 (パス)」を新たな射とみなすんだ。この仕組みを使っ

て、有向グラフ（クイバー）から生成される圏を『自由圏』と呼ぶぜ。

定義 4 (自由圏). 有向グラフ G に対して、 G 上の自由圏 $F(G)$ を次のように定める。

- 対象： G の頂点
- 射：辺を有限個つないだ道
- 恒等射：長さ 0 の道
- 合成：道の連結

魔理沙：有向グラフと圏の繋がりが分かったところで、『プログラミング』の世界でこれがどう機能しているのかを見てみようぜ。今回使うのは『TypeScript（タイプスクリプト）』という、データの『型（Type）』を厳密に扱うプログラミング言語だ。

```
// string を number に変換する
function getLen(str: string): number {
    return str.length;
}

// number を boolean に変換する
function isLong(num: number): boolean {
    return num >= 5;
}

// string -> number -> boolean
const result: boolean = isLong(getLen("hello"));
```

霊夢：最初の関数の出口が `number` で、次の関数の入口も `number` だから合成できるのね。

魔理沙：その通りだぜ。圏論の $f: A \rightarrow B$ と $g: B \rightarrow C$ から $g \circ f: A \rightarrow C$ を作る話そのものだ。

逆に、型の頂点（機械の入り口と出口の形）が一致しない場合は合成が定義できないんだ。例えば、機械の順番を逆に繋ぐとどうなると思う？

```
// getLen は string を受け取りたいが、isLong(5) は boolean を返す
const bad = getLen(isLong(5)); // 型エラー
```

霊夢：えっと、`isLong` の出口からは `boolean`（Yes/No の札）が出てくるのに、次の `getLen` の入り口は `string`（文字列）しか受け付けられないから……材料がつかえて機械が止まっちゃうわね。

魔理沙：大正解だ。辺同士を繋げることができないから、プログラミング言語の側が『これ以上進めませんよ』と警告を出してくれる。これがコンパイルエラーの正体なんだぜ。

霊夢：ただの文字の羅列に見えてたエラーが、実は『矢印がうまく繋がっていませんよ』っていう圏論的なお知らせだったのね！

魔理沙：そうなんだ。プログラミングにおいて、辺の『終点の型』と『次の辺の始点の型』が一致していることが、そのまま辺の合成可能性（計算が正しく連続できるか）に対応しているんだぜ。

注意 (型チェックについて). 型チェックは、射の始域・終域が合っているかを確認する操作として圏論的に解釈できる。ただし、実際のコンパイラが行っている内部処理のすべてが圏論の言葉で説明できるわけではない。

3 関手と自然変換

霊夢：圏論の矢印を引けることは、プログラムが正常に動くことと同じという訳ね。これなら私も、明日には世界を牛耳る天才ハッカーになれそうだわ！

魔理沙：気が早いぜ霊夢。世の中にはもっと複雑な構造を持った言語がたくさんあるんだ。それらを理解するためには、さらに圏論の概念を深掘りしていく必要があるんだぜ。次は圏と圏をつなぐ関手を定義しよう。

定義 5 (関手 (Functor)). 圏 $\mathcal{C}$ から圏 $\mathcal{D}$ への関手 F とは、以下の対応を与えるものである。

1. $\mathcal{C}$ の各対象 X に対し、 $\mathcal{D}$ の対象 $F(X)$ を対応させる。
2. $\mathcal{C}$ の各射 $f: X \rightarrow Y$ に対し、 $\mathcal{D}$ の射 $F(f): F(X) \rightarrow F(Y)$ を対応させる。

さらに、この対応は以下の2つの条件（関手性）を満たす必要がある。

- **恒等射の保存**：各対象 X について、 $F(\text{id}_X) = \text{id}_{F(X)}$
- **合成の保存**： $f: X \rightarrow Y, g: Y \rightarrow Z$ について、 $F(g \circ f) = F(g) \circ F(f)$

霊夢：関手っていうのは、圏から圏へと丸ごと移し替えるようなものなのね。でも、これってプログラミングに関係あるの？

魔理沙：大ありだぜ。プログラミングでは、配列の `map` が関手の例になるぜ。

```
const numbers: number[] = [1, 2, 3];

function numToString(n: number): string {
  return n + "番";
}

const strings: string[] = numbers.map(numToString);
// ["1番", "2番", "3番"]
```

魔理沙：このように、対象 (`number`) を別の対象の箱 (`number[]`) に移し、さらに射 (`number -> string`) も、箱同士の射 (`number[] -> string[]`) に持ち上げている。これがプログラミングにおける「関手 (Functor)」の動きなんだぜ。

注意 (関手法則). 配列が関手であるというためには、単に `map` が存在するだけでは不十分で、次の法則を満たす必要がある。

$$xs.map(id) = xs$$

および

$$xs.map(x \Rightarrow g(f(x))) = xs.map(f).map(g).$$

これは、副作用を持たない純粋関数を扱っている限り、成り立つ。ただし、副作用などを含めた関数を扱う場合はその限りではない。

魔理沙：そういうことだ。続いて自然変換の定義も見よう。

定義 6 (自然変換). 関手 $F, G: \mathcal{C} \rightarrow \mathcal{D}$ の間の自然変換 $\alpha: F \Rightarrow G$ とは、各対象 X に対して射

$$\alpha_X: F(X) \rightarrow G(X)$$

を与え、任意の射 $f: X \rightarrow Y$ について次の図式を可換にするものである。

$$\begin{array}{ccc} F(X) & \xrightarrow{\alpha_X} & G(X) \\ F(f) \downarrow & & \downarrow G(f) \\ F(Y) & \xrightarrow{\alpha_Y} & G(Y) \end{array}$$

すべての α_X が同型射であるとき、 α を自然同型という。

注意 (自然変換の言い方). 自然変換は「圏そのものを変換するもの」ではなく、同じ始域・終域を持つ関手同士をつなぐ射である。

4 表現可能関手と極限

魔理沙：次に、圏論を厳密に扱うために必要なサイズの話をしていくぜ。

定義 7 (小圏と局所小圏). 対象の集まりも射の集まりも、どちらも「集合」に収まるサイズの圏を**小圏** (small category)、対象全体は巨大すぎて集合はないが、任意の対象 A, B について $\mathcal{C}(A, B)$ が集合である圏を**局所小圏** (locally small category) という。

霊夢：集合に収まらないほど巨大なものがあるの？

魔理沙：ああ。「この世のすべての集合」みたいなものを集めると、パラドックスが起きて計算が壊れちゃうんだ。だから私たちがプログラミングなどで扱う圏は、安全な『局所小圏』を前提にすることが多いぜ。この前提のおかげで、次の『表現可能関手』が定義できるんだ。

定義 8 (表現可能関手). 局所小圏 $\mathcal{C}$ から集合の圏 **Set** への反変関手

$$F : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set}$$

が、ある対象 $a \in \mathcal{C}$ について

$$F \cong \mathcal{C}(-, a)$$

と自然同型であるとき、 F は表現可能であるという。つまり、表現可能であるとは、一見よく分からない **Set** への関手が、実は圏 $\mathcal{C}$ の中にある一つの対象 a と、そこへ向かう射によって再現できることを意味する。

定義 9 (離散圏). 対象の集まりからなり、射が恒等射のみである小圏のことを**離散圏** (discrete category) という。

魔理沙：自然数全体を対象とし、恒等射だけを持つ離散圏 $\mathbb{N}$ を考える。

このとき、 $\mathbb{N}(m, n)$ は次のように定まる。

$$\mathbb{N}(m, n) = \begin{cases} \{*\} & (m = n), \\ \emptyset & (m \neq n) \end{cases}$$

したがって、表現可能関手 $\mathbb{N}(-, n)$ は、 n 番目だけが一点集合で、それ以外は空集合であるような列として見られる。

霊夢：ただの対象が、関手という別の姿に綺麗に変換できるってことね。

魔理沙：その通りだ。次はこの普遍的な性質を一般化した極限という概念を見ていくぜ。

定義 10. 関手 $F : \mathcal{J} \rightarrow \mathcal{C}$ があるとき、対象 x から F の各対象への射の集まり（かつ F の射と可換になるもの）を「 x から F への錐 (cone)」と呼ぶ。

定義 11 (極限). $F : \mathcal{J} \rightarrow \mathcal{C}$ への錐の圏の終対象の頂点を $\varprojlim F$ と書き、錐 $(\varprojlim F, \phi)$ を F の極限 (limit) という。また、このときの ϕ を標準射影 (canonical projection) という。

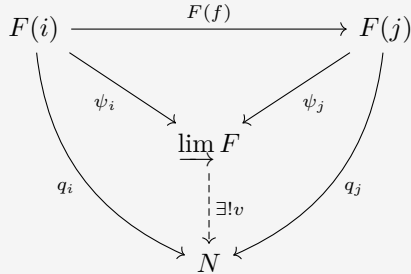
極限は普遍性を持つため、任意の錐 N とその射影 q_i, q_j が与えられたとき、以下の図式を可換にするような射 $u : N \rightarrow \varprojlim F$ が唯一つ存在する。

$$\begin{array}{ccc} & N & \\ q_i \swarrow & \downarrow \exists! u & \searrow q_j \\ & \varprojlim F & \\ \pi_i \swarrow & & \searrow \pi_j \\ F(i) & \xrightarrow{F(f)} & F(j) \end{array}$$

注意 (余極限). $F : \mathcal{J} \rightarrow \mathcal{C}$ からの錐の圏の始対象の頂点を $\varinjlim F$ と書き、錐 $(\varinjlim F, \psi)$ を F の余極限 (colimit) という。また、このときの ψ を標準入射 (canonical inclusion) という。

余極限も同様に普遍性を持つため、任意の余錐 N に対して以下の図式を可換にす

る射 $v : \varinjlim F \rightarrow N$ が唯一つ存在する。



余極限は「錐の圏」ではなく、正確には余錐の圏の始対象として定義する。

5 デカルト閉圏とプログラミング

魔理沙：ここからいよいよ、デカルト閉圏のパーツを揃えていくぜ。まずは終対象・始対象、積・余積、指数対象を定義しよう。これらはプログラミングの型システムに直結する重要な要素だ。

定義 12 (終対象). 圏 $\mathcal{C}$ の対象 1 が**終対象** (terminal object) であるとは、 $\mathcal{C}$ の任意の対象 a に対して、 a から 1 への射 $u : a \rightarrow 1$ がただ一つだけ存在することである。図式で表すと以下ようになる。

$$a \dashrightarrow^{\exists! u} 1$$

注意 (始対象について). 圏 $\mathcal{C}$ の対象 0 が始対象であるとは、 0 から $\mathcal{C}$ の任意の対象 a への射 $v : 0 \rightarrow a$ がただ一つだけ存在することである。図式で表すと以下ようになる（終対象の矢印を逆向きにした形になっていることに注目してほしい）。

$$0 \dashrightarrow^{\exists! v} a$$

定義 13 (積). 圏 $\mathcal{C}$ の対象 a, b に対する**積** (product) とは、 $a \times b$ と書かれるある対象 $a \times b \in \mathcal{C}$ および射 $\pi_a : a \times b \rightarrow a, \pi_b : a \times b \rightarrow b$ の組であって、任意の対象 x と

射 $f : x \rightarrow a, g : x \rightarrow b$ に対して、以下の図式が可換となるような射 $u : x \rightarrow a \times b$ が唯一つ存在するものである。

$$\begin{array}{ccccc} & & x & & \\ & f \swarrow & \downarrow \exists! u & \searrow g & \\ a & \xleftarrow{\pi_a} & a \times b & \xrightarrow{\pi_b} & b \end{array}$$

注意 (余積). 対象 a, b の余積とは、積の双対概念であり、 $a + b$ と書かれる。プログラミングでは、直和型、ユニオン型、`Either A B` のような型に対応する。

注意 (クラスと積型). タプル、record、構造体は積型の代表例である。Java の通常の class もデータの組を表す用途では積型に似ているが、メソッド、可変状態、継承、null などを含むため、一般には単なる積型とは限らない。

霊夢：クラスって、学校のクラスみたいなもの？

魔理沙：似たようなものだ。例えば『名前 (String 型)』と『年齢 (number 型)』という別々のデータがあったとする。Java ではこれを積のルールでまとめて『人間 (Person) クラス』という 1 つの新しいデータ型として扱うことができるんだ。

霊夢：なるほど! 直積が「A かつ B (AND)」で、逆の余積が「A または B (OR)」になるのね。

魔理沙：ああ。そして次が最後のパーツ、『指数対象』だ。

定義 14 (指数対象). 圏 $\mathcal{C}$ が積を持つとする。対象 a, b に対する指数対象とは、対象 b^a と評価射

$$\text{ev} : b^a \times a \rightarrow b$$

の組で、任意の射 $f : x \times a \rightarrow b$ に対して一意な射 $\lambda f : x \rightarrow b^a$ が存在し、次を可換にするものである。

$$\begin{array}{ccc} x \times a & \xrightarrow{(\lambda f) \times \text{id}_a} & b^a \times a \\ & \searrow f & \swarrow \text{ev} \\ & b & \end{array}$$

魔理沙：指数対象は、プログラムでいえば関数型に対応する。

例えば、 b^a は「 a 型の引数を受け取り、 b 型の値を返す関数の型」と見ることができる。

魔理沙：あと、複数の引数を取る関数を、1 つずつ引数を取る関数の連鎖にすることを『カーリー化』というんだ。

霊夢：カーリー化!? 急にスパイシーになってきたわね。天才ハッカーたるもの、美味しいカレーの作り方もマスターしないと!

魔理沙：食べるカレーじゃなくて、数学者の Haskell Curry (ハスケル・カーリー) の名前にちなんだ用語だぜ! せっかくだから、この『カーリー化』がプログラミングでどう役立つのか、コードで見てみよう。

```
// 1. 普通関数: 2つの引数(直積)を同時に受け取って足し算する
function add(a: number, b: number): number {
    return a + b;
}
const result1 = add(5, 3); // 8

// 2. カーリー化された関数: 1つ目の引数を受け取り、「2つ目の引数を足す関数(幕対象)」を返す
function curriedAdd(a: number): (b: number) => number {
    return function(b: number): number {
        return a + b;
    };
}

// カーリー化のメリット: 一部の引数だけを先に渡して、新しい関数(部品)を作れる!
const addFive = curriedAdd(5); // 「5を足す」という専用の関数が完成
const result2 = addFive(3);    // 8
```

霊夢：関数の結果として、また別の関数が返ってきてるわ!

魔理沙：そう。圏論の図式で言えば、直積 ($A \times B$) からの矢印を、指数対象への矢印に変換しているんだ。カーリー化を使えば、既存の関数の一部を固定して、便利な「専用の関数 (この例なら `addFive`)」を後から量産できるんだぜ。

霊夢：汎用的な機械から、特定の作業に特化した専用の機械をポンポン生み出せるのね! 関数をデータとして持ち運べる『指数対象』のルールがあるからこそできる技だわ。

魔理沙：そういうことだ。これで準備はすべて整った。デカルト閉圏の定義を見てみよう。

定義 15 (デカルト閉圏). 圏 C がデカルト閉圏であるとは、次を満たすことである。

1. 終対象を持つ。

2. 任意の有限積を持つ。特に任意の対象 a, b の積 $a \times b$ を持つ。
3. 任意の対象 a, b について指数対象 b^a を持つ。

魔理沙：集合の圏 **Set** はデカルト閉圏の代表例だぜ。

終対象は一点集合、積は直積集合、指数対象 B^A は A から B への写像全体の集合だぜ。

霊夢：これは、Java だとどうなるのかしら？

魔理沙：デカルト閉圏的な構造を近似すると、次のようにコードを書くことができるぜ。

```
// Java には純粋な Unit 型がないので、空の record で近似する
record Unit() {}

// 積型に近い record
record Pair<A, B>(A first, B second) {}

public class CCCExample {
    public static void main(String[] args) {
        Pair<String, Integer> myData = new Pair<>("Hello", 42);

        // 指数対象に対応する関数型の近似
        java.util.function.Function<String, Integer> lengthFunc =
            str -> str.length();

        // 評価射 ev の近似
        Integer result = lengthFunc.apply(myData.first());
        System.out.println("文字数は: " + result);
    }
}
```

注意 (Java の void と Unit). Java の void は値を持つ型ではなく、「値を返さない」ことを表す戻り値指定である。終対象に対応させるなら、Unit 型のような一点型を考えるのが自然である。上の record Unit() はあくまで近似である。

霊夢：おお!なんか今まで習った難しい言葉が、全部スッキリしたコードにまとまっているわ!

魔理沙：そうだろ? ‘Unit’ が終対象、‘Pair’ が積、そして ‘Function’ が指数対象として機能しているんだ。

この論理学・圏論・プログラミングの対応関係のことを、専門用語で『カリー＝ハワード＝ランベック同型対応』と呼ぶんだぜ。

霊夢：カレー・ハワード・ランベック……なんだか強そうな必殺技みたいね! 私もその同型対応の力を使って、天才ハッカーへの第一歩として、まずは明日の夕飯を華麗にカレー（カレー）にしてみせるわ!

魔理沙：ハッカー設定とダジャレが渋滞してるぜ……

※ ここから加筆部分です。

論理	プログラミング	圏論
命題	型	対象
証明	プログラム	射
$\top$	Unit 型	終対象
$A \wedge B$	積型	積
$A \vee B$	直和型	余積
$A \Rightarrow B$	関数型	指数対象

魔理沙：この表からプログラムを書くことは証明を書くことに似ていることがわかると思うぜ。

関連する概念として、トポス理論があるんだぜ。

6 トポス理論との関係

霊夢：ここまですでデカルト閉圏が出てきたけど、トポス理論はどう関係するの？

魔理沙：ざっくり言えば、トポスは「集合の圏 **Set** のように振る舞う圏」だ。デカルト閉圏に加えて、部分集合や真理値を扱う構造を持つんだぜ。

定義 16 (部分対象). 圏 $\mathcal{C}$ の対象 X の部分対象とは、 X へのモノ射 $m : A \hookrightarrow X$ を同型を除いて考えたものである。集合の圏では、これは通常の部分集合 $A \subseteq X$ に対応する。

定義 17 (部分対象分類子). 有限極限を持つ圏 $\mathcal{E}$ において、対象 Ω と射 $\text{true} : 1 \rightarrow \Omega$ が部分対象分類子であるとは、任意のモノ射 $m : A \hookrightarrow X$ に対して一意な射

$\chi_m : X \rightarrow \Omega$ が存在し、次の図式が引き戻しになることである。

$$\begin{array}{ccc} A & \longrightarrow & 1 \\ m \downarrow & & \downarrow \text{true} \\ X & \xrightarrow{\chi_m} & \Omega \end{array}$$

定義 18 (初等トポス). 初等トポスとは、有限極限を持ち、デカルト閉であり、部分対象分類子を持つ圏である。

魔理沙：集合の圏 **Set** では、 $\Omega = \{\text{true}, \text{false}\}$ だ。部分集合 $A \subseteq X$ は、特性関数

$$\chi_A : X \rightarrow \Omega$$

で分類される。

霊夢：つまり、トポスでは「部分集合」や「真理値」まで圏論の言葉で扱えるのね。

6.1 内部論理

魔理沙：トポスの面白いところは、各トポスが内部論理を持つことだ。一般のトポスの内部論理は古典論理ではなく、直観主義論理になることが多い。つまり、排中律 $P \vee \neg P$ が常に成り立つとは限らない。

論理的概念	トポスでの表現
命題	$1 \rightarrow \Omega$
述語 $P(x)$	$X \rightarrow \Omega$
部分集合	部分対象
真理値	Ω

6.2 前層トポスと表現可能関手

魔理沙：小圏 $\mathcal{C}$ に対して、関手圏

$$\mathbf{Set}^{\mathcal{C}^{\text{op}}}$$

を前層圏という。これはトポスの重要な例で、前層トポスと呼ばれる。ここで表現可能関手 $\mathcal{C}(-, a)$ が基本部品として登場する。

霊夢：さっきの表現可能関手が、ここでまた出てくるのね。

魔理沙：そうだ。米田補題や前層トポスは、トポス理論への自然な入口なんだぜ。

7 Cubical Agda：トポスの発想に近い言語

霊夢：Java 以外のプログラミング言語で、トポス理論に近いものはあるの？

魔理沙：代表例としては Cubical Agda がある。Cubical Agda は、依存型プログラミング言語 Agda に Cubical Type Theory の機能を加えた証明支援系だ。計算的 univalence と高次帰納型を扱える点で、Homotopy Type Theory や Univalent Foundations に近い。

7.1 等号をパスとして扱う

```
{-# OPTIONS --cubical #-}

open import Cubical.Core.Primitives

-- A 中の a から b へのパス
PathExample : {A : Type} -> (a b : A) -> Type
PathExample a b = Path A a b
```

魔理沙：普通の等号 $a = b$ を、 a から b への道として扱う。型は単なる集合ではなく、点・線・面を持つ空間のように振る舞うんだ。

7.2 univalence

魔理沙：Cubical Agda の大きな特徴は univalence だ。これは大雑把に言うと、同値な型は等しい型として扱えるという原理だ。

```
{-# OPTIONS --cubical #-}

open import Cubical.Foundations.Univalence
open import Cubical.Foundations.Equiv

uaExample : {A B : Type} -> A  $\simeq$  B -> Path Type A B
uaExample e = ua e
```

霊夢：Java だと、変換関数があっても型そのものが等しいとは言えないものね。

魔理沙：そうだ。Cubical Agda では、同値性と等号が深く結びついている。

7.3 高次帰納型

```
{-# OPTIONS --cubical #-}

data S1 : Type where
```

```
base : S1
loop : Path S1 base base
```

魔理沙：通常のデータ型は値のコンストラクタを持つが、高次帰納型ではパスもコンストラクタにできる。この例では、点 `base` と、そこから自分自身へ戻る道 `loop` によって円を表している。

魔理沙：というわけで、トポス理論や Cubical Agda についても少し触れてみたぜ。
楽しんでくれたのなら幸いだぜ。

霊夢：楽しかったわ。私は Cubical Agda をこれからもっと勉強してみたいと思うわ！

魔理沙：今回は、他のプログラミング言語との関係も掘り下げていく予定だぜ。
それでは、また次回の例大祭で！

参考文献

- [1] 浅芝秀人, グラフ表現で可視化する 圏論, 共立出版
- [2] 梶浦宏成, SGC ライブラリ 75 『数物系のための圏論』, サイエンス社
- [3] 中村 晃 一, Topos Theory(圏 論), <https://nineties.github.io/topos-theory/category>
- [4] 圏論/沓大整域 https://alg-d.com/math/kan_extension
- [5] Tom Lenster, Basic Category Theory, <https://arxiv.org/pdf/1612.09375>
- [6] 中山清喬, 国本大悟, スッキリわかる Java 入門 第4版, インプレス
- [7] 長谷川真人, プログラミング言語の意味論と圏論, <https://www.kurims.kyoto-u.ac.jp/~kenkyubu/kokai-koza/H28-hasegawa.pdf>